SCIKIT-LEARN MODELLING

TUTORIAL 5

(Model Training, Evaluation & Inferencing)

MARTIN MURIITHI

This document contains Hyperlinks



[https://numpy.org]



[https://matplotlib.org]



[https://seaborn.pydata.org]



[https://scikit-learn.org/stable/]



[https://pandas.pydata.org/docs/]

Official respective documentation linked above



Objectives

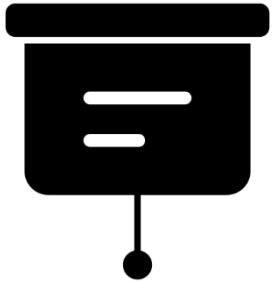
1. Feature pre-processing & model pipelines
2. Model training
3. Hyperparameter tuning
4. Model evaluation
5. Inferencing
6. Model persistence and loading
7. Overview on Model Deployment

Hyper-link included below



Resources...

1. Please note that all the resources for this entire tutorial (code reference in Jupyter Notebook, dataset, slides, question, saved model state and deployment files) are included on my GitHub repository [linked here]
2. If possible, please refer to these slides and the Jupyter Notebook side by side



Introduction

ℹ Some important points to consider before we begin coding

In this tutorial lesson…

We shall:

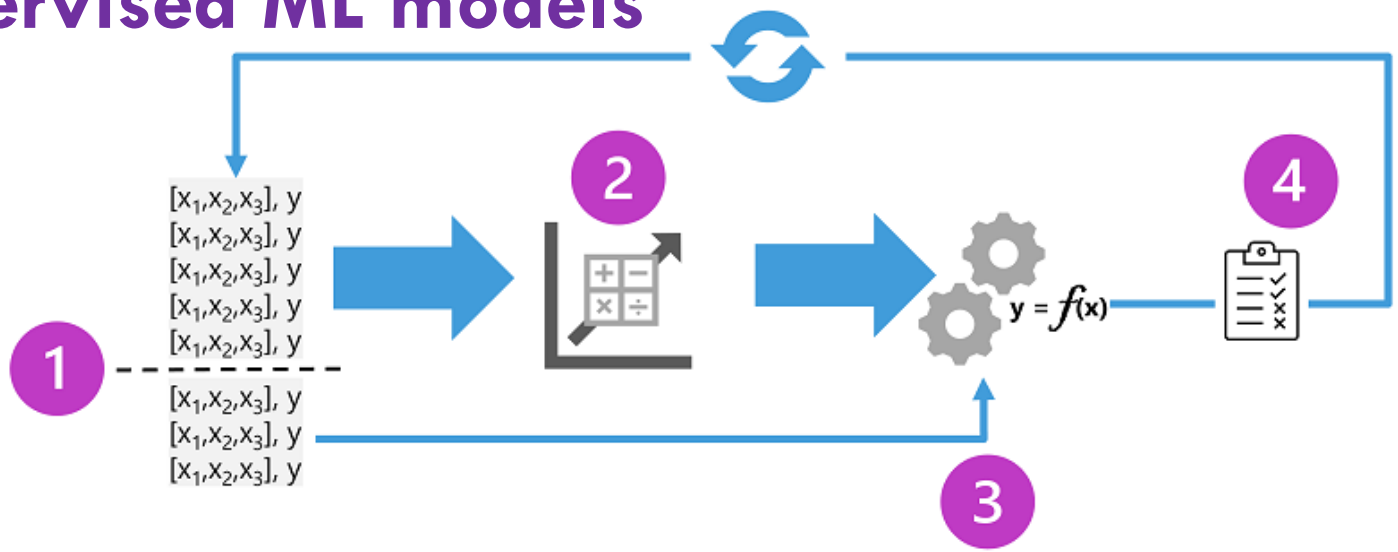
1. Pick off where we left in the previous tutorial…
2. Plug a predictor / ML algorithm (KNN Classifier) into a pipeline
3. Perform simple hyperparameter tuning and train a model
4. Evaluate the trained model
5. Perform inferencing on the trained model
6. Save the model, and illustrate how to reload it

Ideally, one could simply train a model without STEPS 2 & 3, but let's try learning some best practices here, right?

Let’s quickly go over a few key points from the previous tutorial

Source: Microsoft Learn

Key elements of the training process for Supervised ML models



1. Split the data randomly into a training and test / validation set. The model will only be trained using the training set
2. Use an algorithm to fit the training data to the model. In the case of regression, a regression algorithm is used. For classification, a classification algorithm is used
3. Perform inference on the trained model using the held back validation data
4. Compare the known actual / true labels in the validation dataset to the labels the model predicted, in a process known as model evaluation, using evaluation metrics

ⓘ We've already done STEP 1 in the previous tutorial. Hence, we shall do 2, 3 & 4

Hyper-links included below

Recall…

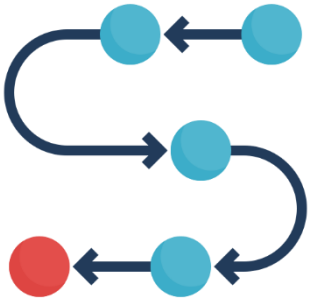
1. A scikit-learn `Pipeline` allows us to chain multiple data pre-processing steps, **and an optional final Machine Learning algorithm (or *predictor* , – as sklearn calls them)** into a **single** unified **object**.
2. Therefore, we can bundle preprocessing & modelling steps into a single `Pipeline` object, ensuring that the exact same steps are applied during training and inference
3. Our features so far have been numerical; hence, we shall scale them using `StandardScaler` as illustrated in the previous tutorial
4. Next, we shall use a `KNeighborsClassifier` algorithm to train a classification model
5. To get some intuition about the KNN algorithm, watch the YOUTUBE video linked **[here]**

Hence, our model Pipeline shall contain a StandardScaler and KNN Classifier



Let's code it out!

ℹ The dataset for this coding tutorial has been provided
(it's the same as the last tutorial's dataset)
Also, I'll be using Google Colab for this tutorial.
Therefore, the file paths and output will definitely look different



Data pre-processing

+

Modelling pipeline

For our case, we'll define a Pipeline with one preprocessing step (scaling) and a classifier algorithm (KNN)

New Jupyter Code Cell

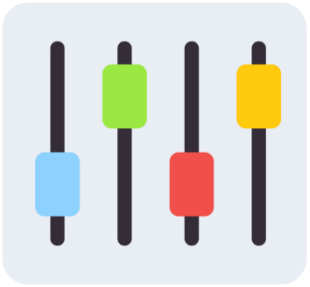
```
# imports
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
```

New Jupyter Code Cell

```
# pipeline object
pipe = Pipeline(steps=[
   #(step name, estimator / transformer)
    ('scaler', StandardScaler()),   # step 1: scaling
    ('knn', KNeighborsClassifier()) # step 2: KNN Classifier
])

# display pipeline as diagram
pipe
```

1. In the first cell, we import necessary sklearn classes as shown
2. Then, we create a Pipeline object named pipe, that has two steps defined: step 1 as scaling step; followed by KNN classifier



Hyper-parameter tuning



Note…

Hyper-links included below

1. A **hyperparameter** is a configuration setting defined by the user ***before*** training a Machine Learning Model, to control the learning process itself. In our case for example, this will be `n_neighbors` in the `KNeighborsClassifier` object
2. **Hyperparameter tuning** is the process of optimizing these settings (hyperparemeters) to improve model performance
3. sklearn provides tools for hyperparameter tuning, such as `GridSearchCV` and `RandomizedSearchCV`.

There's also an entire library dedicated to hyperparameter optimization called optuna, but we won't get into that



GridSearchCV

1. `GridSearchCV` is a tool that performs exhaustive search over specified parameter values for an estimator (a `Pipeline` object is considered an estimator)
2. The parameters of the estimator are optimized by cross-validated grid-search over a parameter grid
3. By default, `GridSearchCV` refits the estimator provided using the best-found parameters on the whole dataset
4. To set or refer to a parameter of a step inside a `Pipeline`, use the *step name* and *parameter name* separated by a double underscore, `__`, for instance: `knn__n_neighbors` in our case
5. Read more in the User Guide

In our case, we'll be trying to find the optimal value of `n_neighbors` for our model pipeline, `pipe`. So, we'll use `GridSearchCV` to search through potential values of `n_neighbors`, and return an estimator fitted using the best-found value on the entire dataset passed

New Jupyter Code Cell

```
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import StratifiedKFold
```

New Jupyter Code Cell

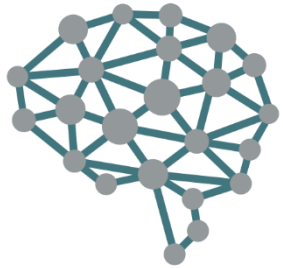
```
# Parameters of pipelines can be set or specified
# using '__' separated by parameter name:
params = {
    'knn__n_neighbors' : np.arange(1,6)
}

# instantiate object
model = GridSearchCV(
    estimator=pipe,
    param_grid=params,
    return_train_score = True,
    scoring='accuracy',
    cv=StratifiedKFold(shuffle=True))
```

CONTINUED

Continued

1. We specify a grid of parameters for `GridSearchCV` to search through. In this case, we'll be trying values `[1, 2, 3, 4, 5]` for hyperparameter `n_neighbors` in the `knn` step of the `Pipeline` object, `pipe`
2. We specify `scoring = 'accuracy'` since we want to optimize for a hyperparameter that yields the highest model accuracy
3. We set `return_train_score = True` since we want the `cv_results_` attribute to include training scores
4. For the `cv` parameter, we provide `StratifiedKFold`; a cross-validation iterator in scikit-learn that provides train/test indices to split data, ensuring that each fold maintains the same percentage of samples for each target class as the original dataset



Model Training

The process of 'teaching' an algorithm to recognize patterns in data (correlation between outcomes and influencing variables) so it can make predictions.

New Jupyter Code Cell

```
1 %%time
2 # train the model
3 model.fit(X_train.values, y_train.values)
```

1. Here, we run fit with all set of parameters
2. We pass `X_train.values` and `y_train.values` to avoid getting a warning message when we later run inference on the model (otherwise passing `X_train` and `y_train` would have been fine). Recall that calling `.values` on a pandas DataFrame returns a numpy array corresponding to the data. The data and its dimensions remain unchanged.
3. `%%time` is a Jupyter Notebook magic command to measure the time it takes to run the code cell
4. `GridSearchCV.fit()` returns an instance of the estimator with the best-found parameters fitted on the entire dataset (`X_train.values` and `y_train.values`)



For consistency, we'll also pass numpy arrays to the model during inference time



Visualize model training for various k-values

New Jupyter Code Cell

```
# plug model.cv_results_ dictionary into a DataFrame
results = pd.DataFrame(model.cv_results_)

# display results DataFrame column names
results.columns
```

New Jupyter Code Cell

```
# rename column appropriately
results.rename(columns={'param_knn__n_neighbors': 'k_values'}, inplace=True)

# visualize a subset of the DataFrame
results[['k_values', 'mean_test_score', 'mean_train_score']]
```

1. **Results from fitting a `GridSearchCV` are stored in an attribute `cv_results_` that we can plug right into a DataFrame. In the first code cell, we do just that and assign to a variable `results`**
2. **Then we display all the column names, just to get an idea of the data**
3. **In the second code cell, we rename the column `param_knn__n_neighbors` to `k_values`**
4. **Finally, we display a subset of that DataFrame, but with only 3 columns as shown**

CONTINUED ▶

```
# rename column appropriately
results.rename(columns={'param_knn__n_neighbors': 'k_values'}, inplace=True)

# visualize a subset of the DataFrame
results[['k_values', 'mean_test_score', 'mean_train_score']]
```

	k_values	mean_test_score	mean_train_score
0	1	0.798469	1.000000
1	2	0.791933	0.904453
2	3	0.848203	0.910667
3	4	0.837728	0.892343
4	5	0.866529	0.897906

1. **The model Pipeline `pipe`, was fitted on the data (`X_train.values`, `y_train.values`) with values of `n_neighbors [1,2,3,4,5]` like we'd specified earlier**
2. **Since `GridSearchCV` optimizes by cross-validation, the `mean_test_score` and `mean_train_score` for all folds across every value of `n_neighbors` are returned**
3. **We only subset using these two columns alongside `k_values` since they are the most important for our analysis**

Read more on cross validation [here]

CONTINUED

New Jupyter Code Cell

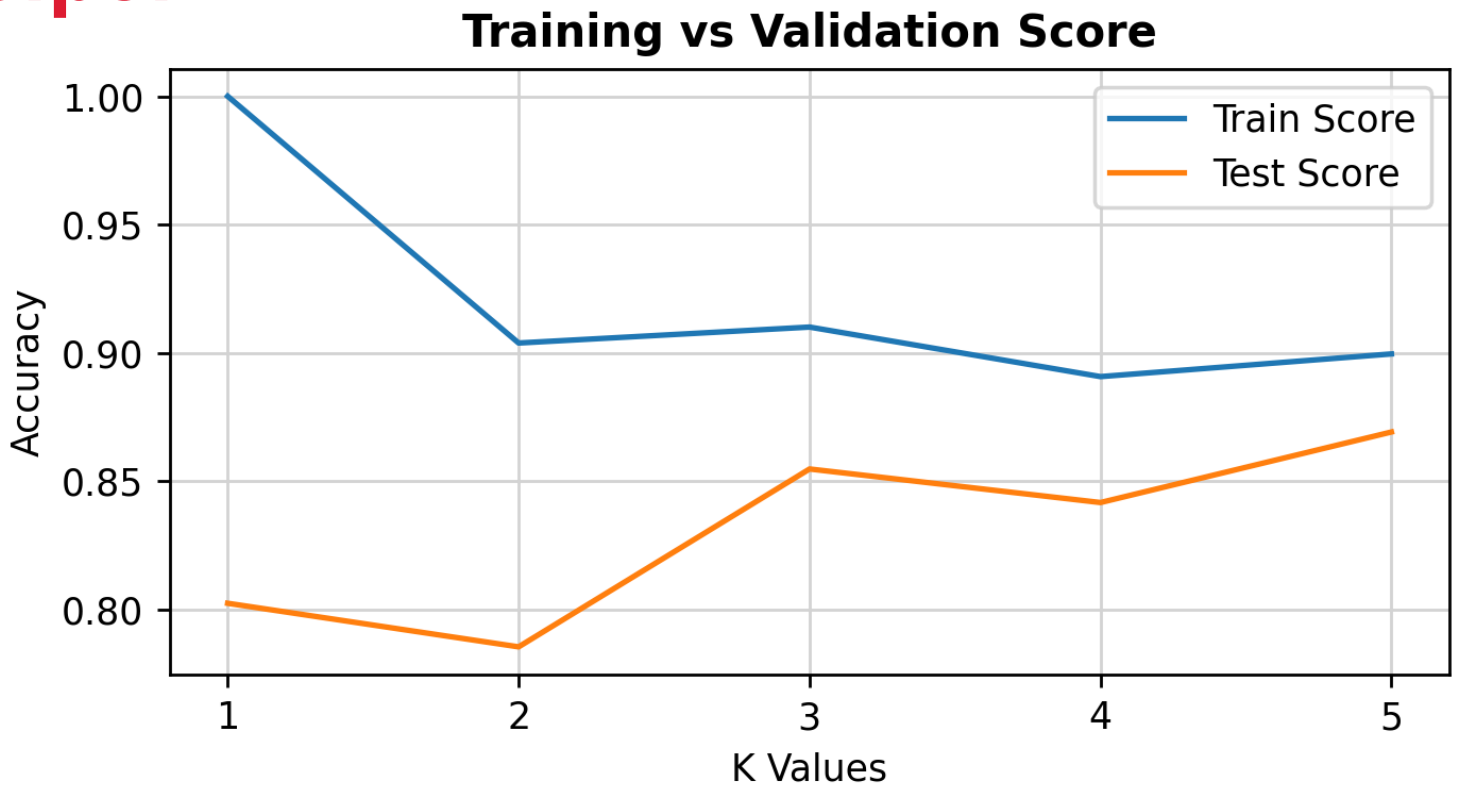
```python
# create figure & subplot
f, ax = plt.subplots(figsize=(5.5,3), layout='compressed')
# make two plots on the Axes
ax.plot(results['k_values'], results['mean_train_score'], label='Train Score')
ax.plot(results['k_values'], results['mean_test_score'], label='Test Score')

ax.set_title('Training vs Validation Score', weight='black')
ax.set_ylabel('Accuracy')
ax.set_xlabel('K Values')
ax.xaxis.set_ticks(np.arange(1, len(params['knn__n_neighbors'])+1))
ax.legend()
ax.grid(axis='both', color='lightgray')
plt.show()
```

1. Next, we'll plot the model training process using matplotlib
2. We return a `figure` and `Axes` object using `plt.subplots()` then plot on the Axes returned
3. We plot `k_values` on the x axis for both the `mean_train_score` and `mean_test_score` columns, assign labels and display legends to distinguish both lines

CONTINUED

Output



1. **NOTE: For now, we'll focus on the Test Score since it measures how well the model generalizes on unseen data. Having both line plots allows us to analyze *variance and bias* (look that up) for the training process**
2. **From the graph, the value of K with the highest `mean_test_score` was `5`**
3. **Recall, that `GridSearchCV.fit()` returns an instance of the estimator with the best-found parameters fitted on the entire dataset (`X_train.values` and `y_train.values`)**

New Jupyter Code Cell

```
1 # best parameter returned
2 model.best_params_
```

New Jupyter Code Cell

```
1 # best estimator returned
2 model.best_estimator_
```

1. **These code cells are just to confirm the best parameters and best estimator returned by `GridSearchCV` after fitting the data**
2. **For the second code cell, expand the diagram, and check the value of `n_neighbors`**



Model Evaluation

The process of using specialized techniques and metrics to assess a machine learning model's performance on unseen data



Note…

Hyper-link included below

1. To assess a Machine Learning Model's performance, we perform inference on data the model has never *seen*, then compare the predictions to ground truth labels
2. In the previous tutorial, we split our `students` dataset into 80% training set and 20% hold-out / validation / test set
3. Recall that the test set is `X_test` for the features, and `y_test` for corresponding labels
4. We shall perform inference on `X_test` to generate predictions, `y_pred`. Then we shall compare `y_pred` to the ground truth outcomes, `y_test`, to assess how well our model performed

We shall evaluate the model using a few (not all) classification metrics. See full list of classification metrics in the sklearn User Guide linked [here]

We need to generate y_pred first...

New Jupyter Code Cell

```
1 # get y_pred
2 y_pred = model.predict(X_test.values)
3
4 # shape of y_pred should be same as y_test:
5 y_pred.shape, y_test.shape
```

1. We've just used our freshly trained model to `predict` on our hold-out dataset, `X_test`, to generate a new set of predictions, `y_pred`
2. From here onwards, we shall compare `y_pred` and `y_test` using various evaluation metrics
3. Check to see that the shape of `y_pred` and `y_test` are the same

(a) accuracy_score

Link to docs included

New Jupyter Code Cell

```python
# import relevant classification metrics
from sklearn.metrics import (accuracy_score,
                             confusion_matrix,
                             ConfusionMatrixDisplay,
                             classification_report)
```

New Jupyter Code Cell

```python
# calculate accuracy
acc = accuracy_score(y_pred=y_pred, y_true=y_test)

# print acc rounded to 2 decimal places
print(f'Model Accuracy: {round(acc, 2)}')
```

1. We import all relevant classification metrics
2. When calling `accuracy_score`, we specify the ground truth labels as `y_test`, and predicted labels as `y_pred`

(b) classification_report

Links to docs included

New Jupyter Code Cell

```python
# generate classification report
cr = classification_report(y_true=y_test, y_pred=y_pred)

# print
print(cr)
```

1. classification_report builds a text report showing the main classification metrics (accuracy, precision, recall, f1 score)
2. When calling classification_report, we specify the ground truth labels as y_test, and predicted labels as y_pred

(c) confusion_matrix

Links to docs included

New Jupyter Code Cell

```python
# generate confusion matrix
cm = confusion_matrix(y_pred=y_pred, y_true=y_test)

# print
print(cm)
```

1. confusion_matrix evaluates classification accuracy by building an error matrix or confusion matrix table. Each row represents an instance of the actual class and each column the instance of a predicted class
2. When calling confusion_matrix, we specify the ground truth labels as y_test, and predicted labels as y_pred

(d) ConfusionMatrixDisplay

Links to docs included

New Jupyter Code Cell

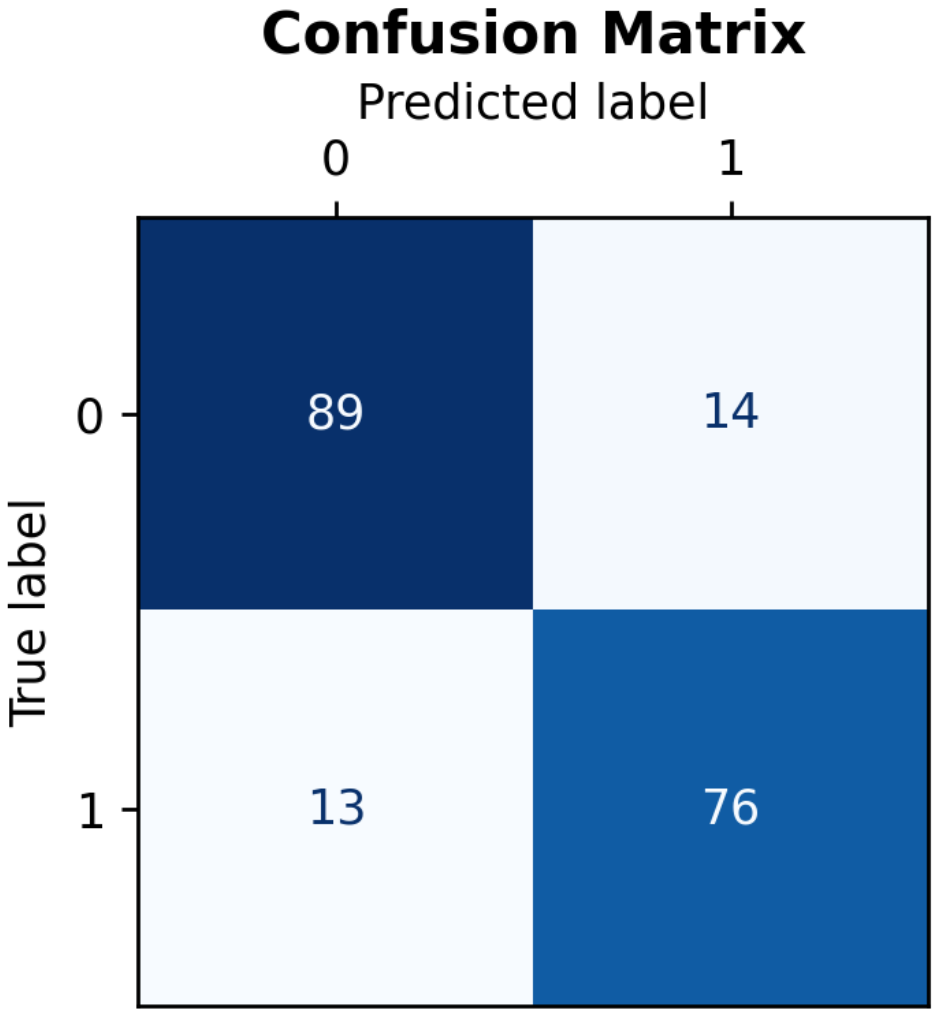
```python
# create figure & Axes
f, ax = plt.subplots(figsize=(3,3))
ConfusionMatrixDisplay.from_predictions(
        y_true=y_test,
        y_pred=y_pred,
        colorbar=False,
        cmap='Blues',
        ax=ax)

ax.set_title('Confusion Matrix', weight='black')
ax.xaxis.tick_top()                  # have ticks appear at the top
ax.xaxis.set_label_position('top') # have xlabel appear at the top
plt.show()
```

1. ConfusionMatrixDisplay.from_predictions is a visualization tool for the confusion matrix given ground truth and predicted labels
2. As we have done so far, we specify the ground truth labels as y_test, and labels outcomes as y_pred

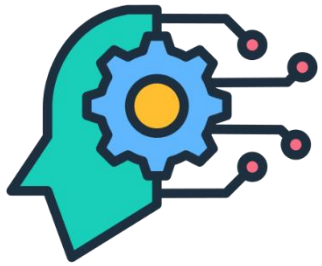
CONTINUED

Output



* **RECALL** that when encoding the label, we specified that **PASS** = **1** & **FAIL** = **0** *

Based on several factors, you might not get a confusion matrix with the exact same values as the one shown above
Read WIKIPEDIA's detailed explanation of confusion matrix linked [here]



Model Inferencing

The process of using a trained machine learning model to make predictions or decisions on new, unseen data in a production environment



Note…

1. Let’s assume that we’re happy with how our model performs based on the classification metrics
2. We can then develop a simple python program that takes in user input: hours studied, attendance percentage, number of assignments completed, past exam score, and uses our model to predict if the student is likely to PASS or FAIL their final examination
3. One could choose to *validate that all user inputs are numeric and within expected range*. You could also *implement error handling*. However, to keep it simple, I won’t be doing that for this tutorial but rather focus on **inferencing**. Try implementing input validation and error handling on your own.



Recall...

Hyper-links included below

1. Using a scikit-learn `Pipeline`, we bundled preprocessing (`StandardScaler`) & modelling (`KNeighborsClassifier`) steps into a single object, ensuring that the exact same steps are applied during training and inference
2. What this means is that we don't have to implement the data scaling step again. The `Pipeline` object, `pipe` that we defined already takes care of that. Data will always be scaled first, before the model performs prediction



We shall wrap the inference logic in a function

New Jupyter Code Cell

```python
# function to perform inferencing
def predict_results():
  # empty list to store student details
  info = []

  print('WELCOME TO THE STUDENT FINAL RESULT PREDICTION SYSTEM\n')

  # get user input, cast to float
  hours = float(input('Insert Hours Studied\n'))
  attendance = float(input('Insert Student Attendance\n'))
  assgn = float(input('Insert No. of Assignments\n'))
  exam_score = float(input('Insert Previous Exam Score\n'))

  # append user input to the empty list
  info.append([hours, attendance, assgn, exam_score])
  # convert list into numpy array
  x_info = np.array(info)

  # perform inference
  if model.predict(x_info).item() == 1:
    prediction = 'PASS'
  else:
    prediction = 'FAIL'

  # display result
  print(f'\nBased on the Data Entered, the student is likely to {prediction}')
```

CONTINUED

Explanation

1. Inside the `predict_results()` function, we declare an empty `list`, `info`, using an empty pair of square brackets, `[]`
2. Next, we take user input and cast it to `float`
3. Next, we `append` the user input as a `list` to the empty `list`, `info` we had declared earlier. NOTE that we keep the order of inputs the same as the order of features we've used throughout training and testing
4. Next, we convert `info` to a `numpy` array using `np.array.` Appending a list to a list ensures we get a 2D array of shape `(1, 4)` when we convert to a `numpy` array. Passing data with less than 2 dimensions to the model throws an error
5. Next, we perform inference on the model using the numpy array of values
6. If the model returns 1, then `prediction = 'PASS'`, otherwise, `'FAIL'` (this was how we had encoded the label earlier in the workflow)
7. Finally, we display a message to the user

Calling `.item()` on a numpy array containing a single element returns that element as a standard python scaler object, e.g `int`, `float`, `str`, e.t.c

Output

New Jupyter Code Cell

```
1 # call the function
2 predict_results()
```

```
# call the function
predict_results()
```

```
WELCOME TO THE STUDENT FINAL RESULT PREDICTION SYSTEM

Insert Hours Studied
8
Insert Student Attendance
75
Insert No. of Assignments
6
Insert Previous Exam Score
87

Based on the Data Entered, the student is likely to PASS
```

Remember to modify the function to perform error handling and input validation since that's not implemented here

This section is completely optional



Model Persistence

The process of saving a trained model's state to a file, allowing it to be loaded later for inference, evaluation, or further training without retraining from scratch

Hyper-links included below

Info

1. To save trained models and later load them, we shall make use of the **`joblib`** library
2. `joblib.dump(value, filename)` persists an arbitrary python object into one file
3. `joblib.load(filename)` reconstructs a python object from a file persisted using `joblib.dump()`

New Jupyter Code Cell

```python
# import joblib
import joblib
```

New Jupyter Code Cell

```python
# define file name
file_name = 'knn_model.joblib'

# save the model
joblib.dump(value=model, filename=file_name)
```

1. We first import `joblib`
2. Then, we define a file name, with optional file path, `file_name`
3. Finally, we call `joblib.dump` and provide `model` for the `value` parameter and the `file_name` for the `filename` parameter



Model Loading

The process of retrieving a saved machine learning model from storage into memory to make predictions, continue training, or deploy it

New Jupyter Code Cell

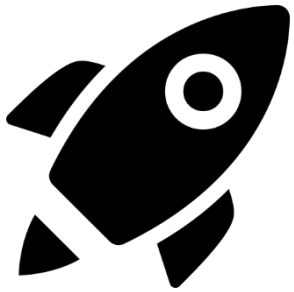
```
# define path to saved file
file_path = '/content/knn_model.joblib'

# load model
loaded_model = joblib.load(filename=file_path)
```

New Jupyter Code Cell

```
# check type of object loaded
type(loaded_model)
```

1. In the first code cell, we define the path to the saved model as `file_path`
2. Then, we call `joblib.load` and pass the `file_path` as `filename` parameter
3. Finally, we check the `type` of the loaded object. This should return `sklearn.model_selection._search.GridSearchCV`



gradio

Hugging Face

Spaces

Model Deployment

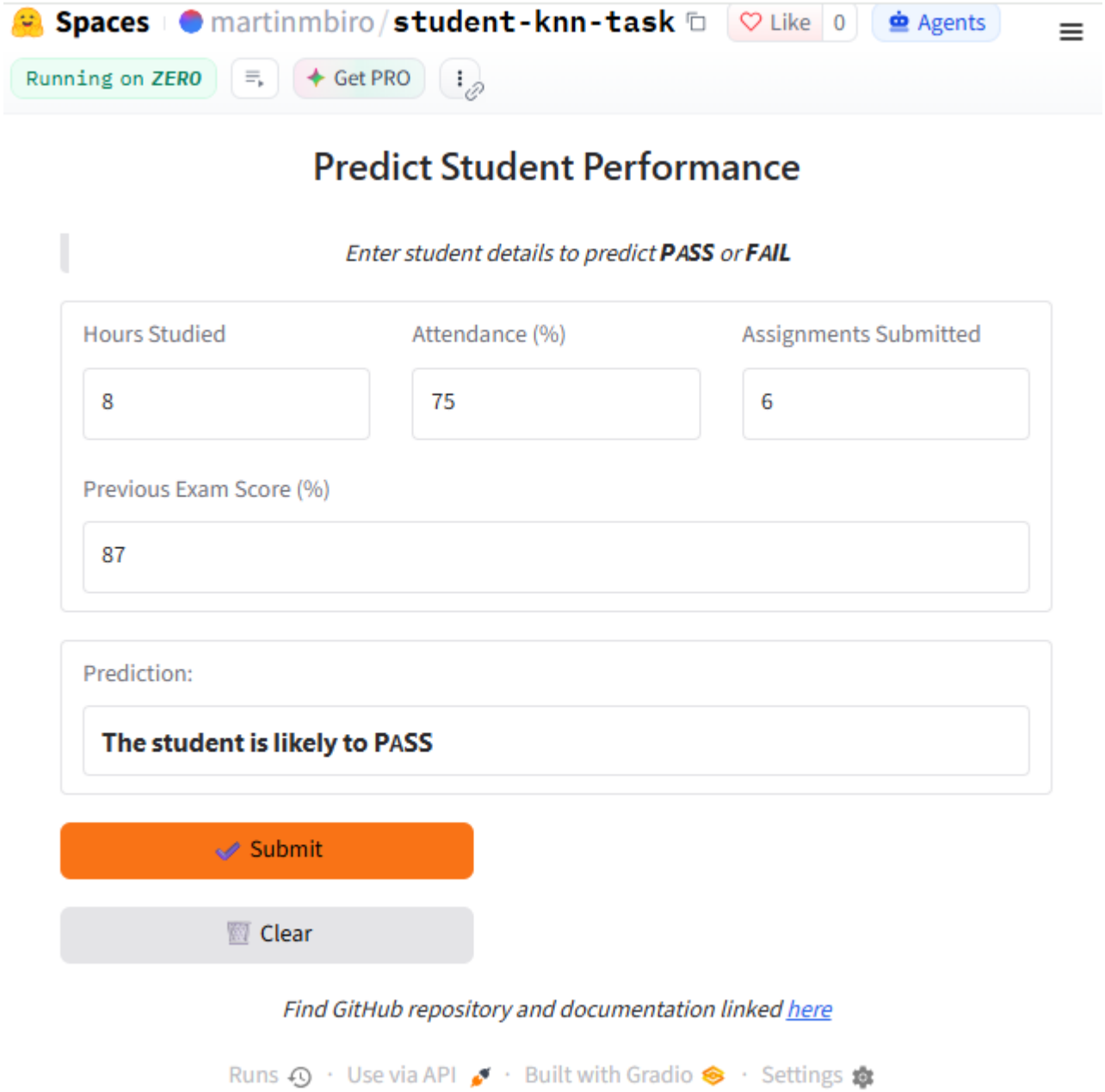
A good place to begin would be with the Hugging Face ecosystem

ℹ Model Deployment is the process of integrating a trained Machine Learning model into a live, real-world production environment so it can process new data and generate practical predictions or decisions

Hyper-links included below

Info

1. If we're to deploy our trained KNN model, we need an interface with **input text boxes**, to enter student data: Hours Studied, Attendance (%), Assignments Submitted, and Previous Examination Score (%). We also need a **button** to submit our data, and an **output text box** to display the prediction.
2. `gradio` is a python library used to create interactive Web Interfaces / Apps for Machine Learning models. The `gradio` code can be run from inside a Jupyter Notebook, or inside a module (a *.py file)
3. These interactive web apps can also be deployed on platforms such as Hugging Face Spaces, SnapDeploy, or Render.
4. With `gradio`, you don't need to know CSS or HTML. Rather, one builds the web interfaces using only Python code. The necessary HTML, CSS and JavaScript is generated automatically behind the scenes.
5. If you wish to go beyond basic layout designs, however, you can still define a custom CSS string and pass it to the gradio launch function.



Web App deployed on my Hugging Face Space linked [here]

Hyper-links included below

Info

1. Model deployment is just hinted and not covered in these slides. That is because it is a different topic entirely, with its own libraries, i.e gradio and huggingface_hub
2. It also requires familiarity with the Hugging Face ecosystem (including platforms such as, Hugging Face Model Hub & Hugging Face Spaces) and GitHub to ease deployment
3. That said, these concepts are covered in the reference Jupyter Notebook and GitHub repository for this tutorial [linked here]. Feel free to follow along!
4. A link to the Web App powered by the trained KNN model is [included here]. Be sure to check that out!

Official python documentation linked [here]



References

Here's a list of recommended references for python, pandas, numpy, scikit-learn, matplotlib, seaborn and all things data science:

- ★ Official pandas documentation linked [here]
- ★ Official numpy documentation linked [here]
- ★ Official matplotlib documentation linked [here]
- ★ Official scikit-learn documentation linked [here]
- ★ Official seaborn documentation linked [here]
- ★ Official gradio documentation linked [here]
- ★ RealPython – website linked [here]
- ★ Medium – website linked [here]

Hyper-links included below

What Next?

1. Over the last two tutorial lessons, we have trained and evaluated a classification model using a KNN algorithm. Try modelling the same dataset using a different algorithm, for instance a tree-based model such as `RandomForestClassifier` or `GradientBoostingClassifier`
2. Also, we have only trained a **binary classification model**. Try looking into multi-class classification and regression



Check out my GitHub portfolio linked [here] to see some cool Data Science and ML projects I have worked on.



Happy Coding!